

MongoDB *Aggregation* Pipeline

You already know find, insert, update. Today you learn the framework that lets the database do the work your JS loops have been doing. Every stage explained with a reason to want it first — never syntax without motivation.

Prerequisite: CRUD basics

mongosh 7.0+

4 collections · 45 documents

The dataset. Four collections — `users`, `posts`, `comments`, `follows`. These exact 45 documents appear in every example below. Every `_id` referenced later (u1–u8, p1–p12, c1–c15, f1–f10) maps to what you see here. Nothing is generated or altered.

users collection – 8 documents

```
[
  { _id: "u1", name: "Aarav Sharma",    username: "aarav_codes",  email: "aarav@mail.co
  { _id: "u2", name: "Diya Patel",      username: "diya.travels", email: "diya@mail.com
  { _id: "u3", name: "Kabir Singh",     username: "kabir_fit",   email: "kabir@mail.co
  { _id: "u4", name: "Ananya Rao",      username: "ananya.eats", email: "ananya@mail.c
  { _id: "u5", name: "Vihaan Gupta",    username: "vihaan_art",  email: "vihaan@mail.c
  { _id: "u6", name: "Ishita Verma",    username: "ishita.codes", email: "ishita@mail.c
  { _id: "u7", name: "Reyansh Joshi",   username: "rey_travels", email: "reyansh@mail.
  { _id: "u8", name: "Myra Kapoor",     username: "myra.fitlife", email: "myra@mail.com
]
```

posts collection – 12 documents

```
[
  { _id: "p1",  userId: "u1", caption: "Deployed my  Aggregation BACKEND WORKSHOP
  { _id: "p2",  userId: "u2", caption: "Sunset in Manali, worth every step",
```

```
[
  { _id: "p3",  userId: "u4",  caption: "Best butter chicken in Indore, change my mind"},
  { _id: "p4",  userId: "u3",  caption: "Leg day never lies",
  { _id: "p5",  userId: "u5",  caption: "New charcoal sketch, took 6 hours",
  { _id: "p6",  userId: "u6",  caption: "Understanding closures finally clicked",
  { _id: "p7",  userId: "u2",  caption: "Road trip through Spiti valley",
  { _id: "p8",  userId: "u8",  caption: "5AM workout hits different",
  { _id: "p9",  userId: "u4",  caption: "Street food tour of Indore, thread",
  { _id: "p10", userId: "u1",  caption: "MongoDB aggregation is actually beautiful",
  { _id: "p11", userId: "u7",  caption: "Backwaters of Kerala at dawn",
  { _id: "p12", userId: "u5",  caption: "Digital art vs traditional, my take",
]
```

comments collection – 15 documents

```
[
  { _id: "c1",  postId: "p1",  userId: "u6",  text: "This is so clean, great work!",  c
  { _id: "c2",  postId: "p1",  userId: "u3",  text: "What stack did you use?",      c
  { _id: "c3",  postId: "p1",  userId: "u2",  text: "Congrats on shipping it",      c
  { _id: "c4",  postId: "p2",  userId: "u1",  text: "Manali is on my list now",      c
  { _id: "c5",  postId: "p2",  userId: "u4",  text: "That sky though",              c
  { _id: "c6",  postId: "p3",  userId: "u1",  text: "Ok now I'm hungry",            c
  { _id: "c7",  postId: "p3",  userId: "u8",  text: "Which place exactly?",         c
  { _id: "c8",  postId: "p3",  userId: "u6",  text: "Facts, best in the city",       c
  { _id: "c9",  postId: "p3",  userId: "u3",  text: "Adding to my list",            c
  { _id: "c10", postId: "p4",  userId: "u8",  text: "Respect the grind",            c
  { _id: "c11", postId: "p5",  userId: "u1",  text: "The shading is insane",         c
  { _id: "c12", postId: "p5",  userId: "u7",  text: "How long did this take?",      c
  { _id: "c13", postId: "p6",  userId: "u1",  text: "Closures took me a month lol",  c
  { _id: "c14", postId: "p6",  userId: "u5",  text: "Underrated concept, good post", c
  { _id: "c15", postId: "p6",  userId: "u8",  text: "Bookmarking this",            c
]
```

follows collection – 10 documents

```
[
  { _id: "f1",  followerId: "u1", followingId: "u2", createdAt: "2024-02-01" },
  { _id: "f2",  followerId: "u1", followingId: "u4", createdAt: "2024-03-15" },
  { _id: "f3",  followerId: "u3", followingId: "u8", createdAt: "2024-04-20" },
  { _id: "f4",  followerId: "u5", followingId: "u2", createdAt: "2024-05-05" },
  { _id: "f5",  followerId: "u6", followingId: "u1", createdAt: "2024-03-16" },
  { _id: "f6",  followerId: "u7", followingId: "u2", createdAt: "2024-01-01" },
  { _id: "f7",  followerId: "u7", followingId: "u4", createdAt: "2024-03-01" },
]
```

```
[
  { _id: "f8", followerId: "u8", followingId: "u4", createdAt: "2024-04-25" },
  { _id: "f9", followerId: "u2", followingId: "u4", createdAt: "2024-02-28" },
  { _id: "f10", followerId: "u4", followingId: "u2", createdAt: "2024-03-05" }
]
```

01

The Problem Aggregation Solves

Why your `find()` + JS loops don't scale

Here's a real question on this dataset: which city has the most verified users, and what's the average follower count of those verified users in that city?

You know CRUD. So you solve it the only way you know — fetch everything, loop in JavaScript, and hope the dataset stays small.

THE PAINFUL WAY

JavaScript — client-side

copy

```
// Step 1: Fetch ALL 8 users from the database
const allUsers = await db.collection('users').find({}).toArray();

// Step 2: Filter verified users in JS — 3 docs pulled for nothing
const verified = allUsers.filter(u => u.isVerified);
// → [u1, u2, u4, u6, u8] — 5 docs kept, 3 wasted bandwidth

// Step 3: Group by city manually
const byCity = {};
for (const u of verified) {
  if (!byCity[u.city]) byCity[u.city] = [];
  byCity[u.city].push(u);
}
// → { Bhopal: [u1, u6], Indore: [u2, u4, u8] }

// Step 4: Compute count + average per city
const results = Object.entries(byCity).map(([city, users]) => ({
  city,
  verifiedCount: users.length,
  avgFollowers: users.reduce((sum, u) => sum + u.followersCount, 0) / users.length
```

Aggregation BACKEND WORKSHOP

```

    }));
    // → [
    //   { city: "Bhopal", verifiedCount: 2, avgFollowers: 1650 },
    //   { city: "Indore", verifiedCount: 3, avgFollowers: 5566.67 }
    // ]

    // Step 5: Sort by count descending
    results.sort((a, b) => b.verifiedCount - a.verifiedCount);

    // Step 6: Grab the winner
    console.log(results[0]);
    // → { city: "Indore", verifiedCount: 3, avgFollowers: 5566.67 }

```

⚠️ Six steps. 20+ lines. Every document fetched over the wire. If users grows to 10 million, you're pulling 10 million documents to filter 3 million away. The database ships data you throw in the trash.

THE AGGREGATION WAY

Same result. Four stages. The database does all the work — only the final row crosses the network.

mongosh

copy

```

db.users.aggregate([
  { $match: { isVerified: true } },
  { $group: {
    _id: "$city",
    verifiedCount: { $sum: 1 },
    avgFollowers: { $avg: "$followersCount" }
  }},
  { $sort: { verifiedCount: -1 } },
  { $limit: 1 }
])

```

Output

```

[
  { _id: "Indore", verifiedCount: 3, avgFollowers: 5566.666666666667 }
]

```

One document returned. The filtering, grouping, averaging, sorting — all happened inside MongoDB. The aggregation framework is a **pipeline**: you feed it stages, documents flow through each one, and what

comes out the other end is your answer.

02

The Mental Model

Pipeline = array of stages. Output of stage N = input of stage N+1.

KEY CONCEPT

A pipeline is an array. Each element is a **stage** — a single operation like filter, group, or sort. Documents enter the first stage, get transformed, and the entire output of that stage becomes the input to the next. If stage 1 outputs 5 documents, stage 2 receives exactly 5 documents. No more, no less — unless a stage itself changes the count (like `$group` collapsing many into few, or `$unwind` splitting one into many).

● 12 posts enter the pipeline

Full posts collection — all 12 documents flow in.

● `$match: { category: "tech" }`

8 documents dropped. 3 tech posts exit: p1, p6, p10.

● `$sort: { likesCount: -1 }`

Still 3 documents — just reordered: p10 (670), p6 (560), p1 (340).

● `$limit: 1`

2 documents dropped. 1 exits: p10.

● Result: 1 document

Only this 1 document crosses the network to your app.



Order matters. `$match` before `$group` means fewer documents to group. `$limit` after `$sort` means you get the true top N. Swap them and you get garbage. Think of it like a Unix pipe — each stage feeds the next.

03

\$match

Filter documents — the WHERE clause of aggregation

Without it: you pull every document from the collection and discard the ones you don't need in JS.

With 8 users that's fine. With 8 million, your app crashes before the loop finishes.

What it does: passes through only documents that match your criteria. Identical syntax to `find()` query operators. Put it *first* whenever possible — MongoDB can use indexes on `$match`, and earlier filtering means every subsequent stage processes fewer documents.

EXAMPLE 1 — VERIFIED USERS ONLY

mongosh

copy

```
db.users.aggregate([
  { $match: { isVerified: true } }
])
```

Output — 5 documents

```
[
  { _id: "u1", name: "Aarav Sharma", ... isVerified: true, followersCount: 1200 },
  { _id: "u2", name: "Diya Patel", ... isVerified: true, followersCount: 4500 },
  { _id: "u4", name: "Ananya Rao", ... isVerified: true, followersCount: 8900 },
  { _id: "u6", name: "Ishita Verma", ... isVerified: true, followersCount: 2100 },
  { _id: "u8", name: "Myra Kapoor", ... isVerified: true, followersCount: 3300 }
]
```

EXAMPLE 2 — FOOD OR TECH POSTS WITH 800+ LIKES

mongosh

copy

```
db.posts.aggregate([
  { $match: {
    likesCount: { $gt: 800 },
    category: { $in: ["tech", "food"] }
  }}
])
```

 Aggregation BACKEND WORKSHOP

Output — 2 documents

```
[
  { _id: "p3", userId: "u4", caption: "Best butter chicken in Indore, change my mind",
    { _id: "p9", userId: "u4", caption: "Street food tour of Indore, thread",
  ]
```



Always put `$match` as early as possible. MongoDB can use indexes to satisfy a `$match` at the start of a pipeline. Once you run a `$project` or `$group` before `$match`, the optimizer can't use indexes — it has to scan every document. This is the #1 performance mistake in aggregation.

04

\$group

Collapse many documents into fewer — the GROUP BY of MongoDB

Without it: you fetch all posts, loop through them in JS, maintain a dictionary keyed by category, and compute sums/averages yourself. It works, but the database has to ship every document to your app, and your JS is single-threaded.

What it does: groups input documents by a specified key (`_id`) and applies accumulator operators to each group. The output is one document per unique group value.

EXAMPLE 1 — COUNT POSTS PER CATEGORY

mongosh

copy

```
db.posts.aggregate([
  { $group: {
    _id: "$category",
    postCount: { $sum: 1 }
  }}
])
```

Output — 5 documents (one per category)

Aggregation BACKEND WORKSHOP

```
[
  { _id: "tech",    postCount: 3 },
  { _id: "travel",  postCount: 3 },
  { _id: "food",    postCount: 2 },
  { _id: "fitness", postCount: 2 },
  { _id: "art",     postCount: 2 }
]
```

EXAMPLE 2 – TOTAL + AVERAGE LIKES PER CATEGORY

mongosh

copy

```
db.posts.aggregate([
  { $group: {
    _id: "$category",
    totalLikes: { $sum: "$likesCount" },
    avgLikes: { $avg: "$likesCount" },
    postCount: { $sum: 1 }
  }},
  { $sort: { totalLikes: -1 } }
])
```

Output – sorted by totalLikes descending

```
[
  { _id: "food",    totalLikes: 3300, avgLikes: 1650,    postCount: 2 },
  { _id: "travel",  totalLikes: 3290, avgLikes: 1096.67, postCount: 3 },
  { _id: "tech",    totalLikes: 1570, avgLikes: 523.33,  postCount: 3 },
  { _id: "fitness", totalLikes: 990,  avgLikes: 495,    postCount: 2 },
  { _id: "art",     totalLikes: 740,  avgLikes: 370,    postCount: 2 }
]
```



\$group does not preserve document order. The output order is arbitrary unless you add a **\$sort** after. Also, grouping by **null** (**_id: null**) collapses all documents into a single group — useful for counting an entire collection.

\$project

Reshape documents — pick, drop, and compute fields

Without it: every document ships over the wire with all 9 fields, even if your frontend only needs 2. You're paying bandwidth and serialization costs for data you throw away.

What it does: passes through documents with only the fields you specify. Use `1` to include, `0` to exclude. You can also compute new fields — expressions make `$project` the stage where data gets reshaped, not just trimmed.

EXAMPLE 1 — PICK ONLY NAME AND USERNAME

mongosh

copy

```
db.users.aggregate([
  { $project: { _id: 0, name: 1, username: 1 } }
])
```

Output — 8 documents, 2 fields each

```
[
  { name: "Aarav Sharma", username: "aarav_codes" },
  { name: "Diya Patel", username: "diya.travels" },
  { name: "Kabir Singh", username: "kabir_fit" },
  { name: "Ananya Rao", username: "ananya.eats" },
  { name: "Vihaan Gupta", username: "vihaan_art" },
  { name: "Ishita Verma", username: "ishita.codes" },
  { name: "Reyansh Joshi", username: "rey_travels" },
  { name: "Myra Kapoor", username: "myra.fitlife" }
]
```

EXAMPLE 2 — COMPUTE ENGAGEMENT (LIKES + COMMENTS)

mongosh

copy

```
db.posts.aggregate([
  { $project: {
    _id: 1,
    caption: 1,
    engagement: { $add: ["$likesCount", "$commentsCount"] }
  }
}]
```

Aggregation BACKEND WORKSHOP

```
  }}  
  ])
```

Output — 12 documents

```
[  
  { _id: "p1", caption: "Deployed my first MERN app today!", engagement: 343 },  
  { _id: "p2", caption: "Sunset in Manali, worth every step", engagement: 892 },  
  { _id: "p3", caption: "Best butter chicken in Indore...", engagement: 1204 },  
  { _id: "p4", caption: "Leg day never lies", engagement: 211 },  
  { _id: "p5", caption: "New charcoal sketch, took 6 hours", engagement: 432 },  
  { _id: "p6", caption: "Understanding closures finally clicked", engagement: 563 },  
  { _id: "p7", caption: "Road trip through Spiti valley", engagement: 1450 },  
  { _id: "p8", caption: "5AM workout hits different", engagement: 782 },  
  { _id: "p9", caption: "Street food tour of Indore, thread", engagement: 2100 },  
  { _id: "p10", caption: "MongoDB aggregation is actually beautiful", engagement: 673 },  
  { _id: "p11", caption: "Backwaters of Kerala at dawn", engagement: 950 },  
  { _id: "p12", caption: "Digital art vs traditional, my take", engagement: 310 }  
]
```

⚠ You cannot mix 1 and 0 freely. Once you include a field with `1`, all other fields are excluded by default — you can only use `0` to explicitly remove `_id`. The exception: if you set only exclusions (all `0`), the rest come along. This is the most common \$project mistake.

06

\$sort

Order documents by one or more fields

Without it: you fetch all documents, then call `.sort()` in JS on the full array. Fine for 12 posts, catastrophic for 12 million. The database has an internal sort that's dramatically faster.

What it does: reorders all incoming documents. Use `1` for ascending, `-1` for descending. Multiple fields are sorted lexicographically — first field first, ties broken by second, etc.

EXAMPLE 1 — POSTS BY LIKES DESCENDING

mongosh

copy

```
db.posts.aggregate([
  { $sort: { likesCount: -1 } }
])
```

Output – top 5 shown

```
[
  { _id: "p9", likesCount: 2100, caption: "Street food tour of Indore, thread", ... },
  { _id: "p7", likesCount: 1450, caption: "Road trip through Spiti valley", ... },
  { _id: "p3", likesCount: 1200, caption: "Best butter chicken in Indore...", ... },
  { _id: "p11", likesCount: 950, caption: "Backwaters of Kerala at dawn", ... },
  { _id: "p2", likesCount: 890, caption: "Sunset in Manali, worth every step", ... }
]
```

EXAMPLE 2 – USERS BY FOLLOWERS ASCENDING, THEN AGE DESCENDING

mongosh

copy

```
db.users.aggregate([
  { $sort: { followersCount: 1, age: -1 } },
  { $project: { name: 1, followersCount: 1, age: 1, _id: 0 } }
])
```

Output – 8 documents

```
[
  { name: "Vihaan Gupta", followersCount: 150, age: 21 },
  { name: "Kabir Singh", followersCount: 320, age: 28 },
  { name: "Reyansh Joshi", followersCount: 600, age: 26 },
  { name: "Aarav Sharma", followersCount: 1200, age: 22 },
  { name: "Ishita Verma", followersCount: 2100, age: 23 },
  { name: "Myra Kapoor", followersCount: 3300, age: 27 },
  { name: "Diya Patel", followersCount: 4500, age: 25 },
  { name: "Ananya Rao", followersCount: 8900, age: 24 }
]
```



`$sort` has a 100MB memory limit by default. If the pipeline is sorting more data than fits in memory, MongoDB throws an error. Add `{ allowDiskUse: true }` as a second argument to `aggregate()` to let it spill to disk. Better: always `$match` before `$sort` to reduce the input size.

07

`$limit`

Pass through only the first N documents

Without it: you fetch all 12 posts to display a "Top 3" leaderboard. 9 documents crossed the network for nothing.

What it does: passes through at most N documents, drops the rest. Pair with `$sort` to get "top N" — but `$sort` must come *before* `$limit`, or you're limiting random documents then sorting what's left.

EXAMPLE — TOP 3 MOST LIKED POSTS

mongosh

copy

```
db.posts.aggregate([
  { $sort: { likesCount: -1 } },
  { $limit: 3 },
  { $project: { caption: 1, likesCount: 1, _id: 0 } }
])
```

Output — 3 documents

```
[
  { caption: "Street food tour of Indore, thread", likesCount: 2100 },
  { caption: "Road trip through Spiti valley",      likesCount: 1450 },
  { caption: "Best butter chicken in Indore...",   likesCount: 1200 }
]
```

08

\$skip

Discard the first N documents, pass through the rest

Without it: you implement pagination by fetching all documents and slicing the array in JS. Page 1000 means you fetched 10,000 documents to show 10.

What it does: drops the first N documents, passes everything after. Almost always paired with `$limit` for pagination: `$skip` N, then `$limit` pageSize.

EXAMPLE — PAGINATION: PAGE 2 OF POSTS (3 PER PAGE, SORTED BY LIKES)

mongosh

copy

```
db.posts.aggregate([
  { $sort: { likesCount: -1 } },
  { $skip: 3 },           // skip page 1 (3 items)
  { $limit: 3 },         // take page 2
  { $project: { caption: 1, likesCount: 1, _id: 0 } }
])
```

Output — ranked 4th–6th by likes

```
[
  { caption: "Backwaters of Kerala at dawn",      likesCount: 950 },
  { caption: "Sunset in Manali, worth every step", likesCount: 890 },
  { caption: "5AM workout hits different",        likesCount: 780 }
]
```



`$skip` then `$limit`, not the reverse. If you `$limit` first, you keep only the first 3, then skip 3 of those — you get zero. The order is: sort → skip → limit.

09

\$unwind

Explode an array — one output document per element

Without it: you want to count how many posts use the tag "webdev". The tags live inside an array on each post. With `find()`, you'd loop through every post, then loop through every tag, maintaining a tally in JS.

What it does: takes one document with an array of N elements and produces N documents, each with one element from the array. The original array field becomes a scalar. This is the prerequisite for `$group` on array contents.

EXAMPLE 1 – UNWIND TAGS ON POSTS

mongosh

copy

```
db.posts.aggregate([
  { $unwind: "$tags" },
  { $project: { _id: 1, caption: 1, tags: 1 } }
])
```

Output – 32 documents (sum of all tag array lengths)

```
// p1 has 3 tags → 3 documents:
{ _id: "p1", caption: "Deployed my first MERN app today!", tags: "mern" },
{ _id: "p1", caption: "Deployed my first MERN app today!", tags: "webdev" },
{ _id: "p1", caption: "Deployed my first MERN app today!", tags: "nodejs" },

// p6 has 2 tags → 2 documents:
{ _id: "p6", caption: "Understanding closures finally clicked", tags: "javascript" },
{ _id: "p6", caption: "Understanding closures finally clicked", tags: "webdev" },

// ... 32 total documents, one per tag per post
```

EXAMPLE 2 – MOST POPULAR TAGS (UNWIND → GROUP → SORT)

mongosh

copy

```
db.posts.aggregate([
  { $unwind: "$tags" },
  { $group: { _id: "$tags", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

Output — tags used 2+ times

```
[
  { _id: "webdev",    count: 3 }, // p1, p6, p10
  { _id: "travel",   count: 3 }, // p2, p7, p11
  { _id: "food",     count: 2 }, // p3, p9
  { _id: "indore",   count: 2 }, // p3, p9
  { _id: "fitness", count: 2 }, // p4, p8
  { _id: "art",      count: 2 }, // p5, p12
  // ... 18 more tags with count: 1
]
```

⚠ Forgetting `$unwind` before `$group` on array contents is the #1 aggregation bug. If you `$group` on `$tags` without unwinding first, MongoDB treats the entire array as a single key — you'd get one group per unique array combination, not per tag.

10

\$lookup

Join another collection — the SQL LEFT JOIN of MongoDB

Without it: to show a post with its author's name, you fetch the post, extract `userId`, fire a second query to `users`, and stitch them together in JS. For a feed of 50 posts, that's 51 round trips to the database.

What it does: performs a left outer join from the current collection to another collection. For each input document, it finds matching documents in the foreign collection and attaches them as an array.

EXAMPLE 1 — ATTACH AUTHOR INFO TO EACH POST

mongosh

copy

```
db.posts.aggregate([
  { $lookup: {
    from:      "users",
    localField: "userId",
    foreignField: "_id",
    as:        "author"
  }},
```

Aggregation BACKEND WORKSHOP

```
{ $project: { caption: 1, "author.name": 1, "author.city": 1, _id: 0 } }
])
```

Output – 12 documents, first 3 shown

```
[
  { caption: "Deployed my first MERN app today!",    author: [{ name: "Aarav Sharma",
  { caption: "Sunset in Manali, worth every step",    author: [{ name: "Diya Patel",
  { caption: "Best butter chicken in Indore...",      author: [{ name: "Ananya Rao",
    // ... 9 more
]
```

EXAMPLE 2 – EACH USER WITH THEIR POST COUNT

mongosh

copy

```
db.users.aggregate([
  { $lookup: {
    from:        "posts",
    localField:   "_id",
    foreignField: "userId",
    as:          "userPosts"
  }},
  { $project: {
    name: 1,
    postCount: { $size: "$userPosts" },
    _id: 0
  }}
])
```

Output – 8 documents

```
[
  { name: "Aarav Sharma",    postCount: 2 }, // p1, p10
  { name: "Diya Patel",      postCount: 2 }, // p2, p7
  { name: "Kabir Singh",     postCount: 1 }, // p4
  { name: "Ananya Rao",      postCount: 2 }, // p3, p9
  { name: "Vihaan Gupta",    postCount: 2 }, // p5, p12
  { name: "Ishita Verma",    postCount: 1 }, // p6
  { name: "Reyansh Joshi",   postCount: 1 }, // p8
]
```

```
{ name: "Myra Kapoor", postCount: 1 } // p8
]
```

💡 The joined field is always an array, even if only one document matches. Use `$unwind` on the `as` field if you want to flatten it back to a single object. Create an index on the `foreignField` (`userId` on `posts`) — without it, every `$lookup` does a full collection scan.

11

\$graphLookup

Recursive join — traverse relationships to arbitrary depth

Without it: to find "everyone in u2's follower network" (who follows u2, and who follows those followers, and so on), you'd write a recursive function that fires one query per level. For a social graph, that's potentially hundreds of round trips.

What it does: recursively traverses a collection, following a connect-from → connect-to relationship. Starting from a seed value, it finds all reachable documents. Perfect for org charts, category trees, and social graphs.

EXAMPLE — FIND U2'S RECURSIVE FOLLOWER NETWORK

Starting from u2, find everyone who follows u2, then everyone who follows *those* people, and so on — traversing the `follows` collection where `followingId` points to the current user and `followerId` is the next hop.

mongosh

copy

```
db.users.aggregate([
  { $match: { _id: "u2" } },
  { $graphLookup: {
    from: "follows",
    startWith: "$_id", // seed = "u2"
    connectFromField: "followerId", // hop to next level via followerId
    connectToField: "followingId", // match against followingId
    as: "followerNetwork"
  } },
])
```

```
{ $project: { name: 1, networkSize: { $size: "$followerNetwork" }, _id: 0 } }
})
```

Output

```
[
  {
    name: "Diya Patel",
    networkSize: 4,
    // followerNetwork contains 4 follow edges:
    // f1: u1 → u2 (u1 directly follows u2)
    // f4: u5 → u2 (u5 directly follows u2)
    // f6: u7 → u2 (u7 directly follows u2)
    // f5: u6 → u1 (u6 follows u1, who follows u2)
  }
]
```

HOW THE TRAVERSAL WORKS

Level 0: start with "u2". Find follows where `followingId = "u2"` → f1 (u1), f4 (u5), f6 (u7). Three direct followers found.

Level 1: take followerId values {u1, u5, u7}. Find follows where `followingId` ∈ {u1, u5, u7} → f5 (u6 → u1). One new follower: u6.

Level 2: take {u6}. Find follows where `followingId = "u6"` → none. Traversal ends.

Total: 4 edges in the network. The people who (transitively) follow u2: u1, u5, u7, u6.

⚠ **\$graphLookup is expensive.** It's a recursive traversal with no index on the connectToField beyond a basic scan. For deep graphs, add `maxDepth` to cap recursion. For shallow (1-level) joins, use regular `$lookup` — it's faster.

12

\$addFields / \$set

Add new fields while keeping all existing ones

Without it: you fetch documents, loop through them in JS to add computed fields, then use the modified array. The transformation happens outside the database.

What it does: adds new fields (or overwrites existing ones) to every document. Unlike `$project`, all original fields are preserved — you're only adding. `$set` is an alias for `$addFields` (identical behavior, different name).

EXAMPLE 1 — ADD ENGAGEMENT SCORE TO POSTS

mongosh

copy

```
db.posts.aggregate([
  { $addFields: {
    engagement: { $add: ["$likesCount", "$commentsCount"] }
  }},
  { $project: { caption: 1, engagement: 1, _id: 0 } }
])
```

Output — first 4 shown

```
[
  { caption: "Deployed my first MERN app today!",      engagement: 343 },
  { caption: "Sunset in Manali, worth every step",     engagement: 892 },
  { caption: "Best butter chicken in Indore...",      engagement: 1204 },
  { caption: "Leg day never lies",                    engagement: 211 }
]
```

EXAMPLE 2 — ADD FOLLOWER TIER TO USERS

mongosh

copy

```
db.users.aggregate([
  { $addFields: {
    tier: {
      $switch: {
        branches: [
          { case: { $gte: ["$followersCount", 5000] }, then: "mega" },
          { case: { $gte: ["$followersCount", 1000] }, then: "mid" }
        ],
        default: "small"
      }
    }
  }
}]
```

```
}},  
{ $project: { name: 1, followersCount: 1, tier: 1, _id: 0 } }  
)
```

Output — 8 documents

```
[  
  { name: "Aarav Sharma",    followersCount: 1200, tier: "mid" },  
  { name: "Diya Patel",      followersCount: 4500, tier: "mid" },  
  { name: "Kabir Singh",     followersCount: 320,  tier: "small" },  
  { name: "Ananya Rao",      followersCount: 8900, tier: "mega" },  
  { name: "Vihaan Gupta",    followersCount: 150,  tier: "small" },  
  { name: "Ishita Verma",    followersCount: 2100, tier: "mid" },  
  { name: "Reyansh Joshi",   followersCount: 600,  tier: "small" },  
  { name: "Myra Kapoor",     followersCount: 3300, tier: "mid" }  
]
```

13

\$unset

Remove fields — the inverse of \$addField

Without it: you use `$project` with exclusion mode, but `$project` can't also compute new fields in the same stage. You'd need two stages. `$unset` is the clean way to drop fields while keeping everything else.

What it does: removes specified fields from every document. Accepts a single field name string or an array of field names. The opposite of `$addField`.

EXAMPLE — STRIP EMAIL AND ISVERIFIED FROM USERS

mongosh

copy

```
db.users.aggregate([  
  { $unset: ["email", "isVerified"] }  
])
```

Output — first document shown

Aggregation BACKEND WORKSHOP

```
{
  _id: "u1",
  name: "Aarav Sharma",
  username: "aarav_codes",
  age: 22,
  city: "Bhopal",
  joinedAt: "2024-01-15",
  followersCount: 1200
  // email and isVerified are gone, everything else preserved
}
```



Use `$unset` when you want to drop fields but keep everything else. Use `$project` with inclusion mode when you want to keep *only* specific fields. They're inverse operations with different ergonomics.

14

`$replaceRoot` / `$replaceWith`

Replace the entire document with a new shape

Without it: after a `$lookup`, the joined data sits in an array field. You want the joined data to *be* the document, not a nested array. `$project` can rename fields one by one, but can't wholesale replace the document root.

What it does: discards the current document and replaces it entirely with the specified object.

`$replaceWith` is the simpler alias (MongoDB 4.2+) — it takes a single expression that becomes the new document. `$replaceRoot` is the older form using `{ newRoot: ... }`.

EXAMPLE — RESHAPE POSTS INTO A MINIMAL VIEW

mongosh

copy

```
db.posts.aggregate([
  { $replaceWith: {
    id: "$_id",
    text: "$caption",
    popularity: "$likesCount",
    topic: "$category"
  }
}]
```

Aggregation BACKEND WORKSHOP

```
  }}  
  ])
```

Output — first 3 documents

```
[  
  { id: "p1", text: "Deployed my first MERN app today!",      popularity: 340, topic: "MERN",  
    { id: "u1", text: "I love MERN", popularity: 100, topic: "MERN" },  
    { id: "u2", text: "MERN is the best", popularity: 240, topic: "MERN" }  
  },  
  { id: "p2", text: "Sunset in Manali, worth every step",      popularity: 890, topic: "Travel",  
    { id: "u3", text: "Manali is beautiful", popularity: 100, topic: "Travel",  
      { id: "u4", text: "I love MERN", popularity: 100, topic: "MERN" }  
    },  
    { id: "u5", text: "MERN is the best", popularity: 240, topic: "MERN" }  
  },  
  { id: "p3", text: "Best butter chicken in Indore...",        popularity: 1200, topic: "Food",  
    { id: "u6", text: "Indore is the best", popularity: 100, topic: "Food",  
      { id: "u7", text: "I love MERN", popularity: 100, topic: "MERN" }  
    },  
    { id: "u8", text: "MERN is the best", popularity: 240, topic: "MERN" }  
  }  
]
```

A common pattern: `$replaceRoot` with `$mergeObjects` to flatten a looked-up sub-document into the top level:

mongosh — flatten lookup result

copy

```
db.posts.aggregate([  
  { $lookup: {  
    from: "users", localField: "userId", foreignField: "_id", as: "author"  
  }},  
  { $unwind: "$author" },  
  { $replaceRoot: {  
    newRoot: { $mergeObjects: ["$author", "$$ROOT"] }  
  }},  
  { $unset: ["author", "userId"] }  
])  
  
// Now each post document has author fields (name, city, etc.)  
// merged in at the top level, no nesting.
```

15

\$out

Write pipeline results to a new collection — destructive

Without it: you run an aggregation, get results in your app, then loop through them calling

`insertMany()` to save to another collection. Two round trips and a lot of JS.

Aggregation BACKEND WORKSHOP

What it does: writes the output of the pipeline to a collection. If the collection exists, it's **dropped** and **replaced** entirely. Must be the last stage in the pipeline.

EXAMPLE — SAVE CATEGORY STATS TO A NEW COLLECTION

mongosh

copy

```
db.posts.aggregate([
  { $group: {
    _id: "$category",
    totalLikes: { $sum: "$likesCount" },
    avgLikes: { $avg: "$likesCount" },
    postCount: { $sum: 1 }
  }},
  { $out: "categoryStats" }
])
```

categoryStats collection — after running the above

```
[
  { _id: "tech",    totalLikes: 1570, avgLikes: 523.33, postCount: 3 },
  { _id: "travel", totalLikes: 3290, avgLikes: 1096.67, postCount: 3 },
  { _id: "food",   totalLikes: 3300, avgLikes: 1650,   postCount: 2 },
  { _id: "fitness", totalLikes: 990,  avgLikes: 495,    postCount: 2 },
  { _id: "art",    totalLikes: 740,  avgLikes: 370,    postCount: 2 }
]
```

⚠️ **\$out** destroys the target collection if it exists. All existing documents are gone — this is not a merge. If you need to add to or update an existing collection, use **\$merge** instead. Also, **\$out** can only write to the same database, and the output collection cannot be sharded.

16

\$merge

Write pipeline results to a collection — non-destructive, upsert-capable

Without it: you compute daily stats, fetch existing documents from the stats collection, and then update or insert each one individually. A cron job becomes 50 database calls.

What it does: writes pipeline output to a collection with fine-grained control: insert new, replace existing, merge fields, or keep existing. Unlike `$out`, it preserves existing documents. Can target a different database.

EXAMPLE — MERGE USER COUNT PER CITY INTO CITYSTATS, REPLACING ON MATCH

mongosh

copy

```
db.users.aggregate([
  { $group: {
    _id: "$city",
    userCount: { $sum: 1 },
    avgFollowers: { $avg: "$followersCount" }
  }},
  { $merge: {
    into: "cityStats",
    on: "_id",
    whenMatched: "replace",
    whenNotMatched: "insert"
  }}
])
```

cityStats collection

```
[
  { _id: "Bhopal", userCount: 3, avgFollowers: 1206.67 },
  { _id: "Indore", userCount: 3, avgFollowers: 5566.67 },
  { _id: "Pune", userCount: 2, avgFollowers: 375 }
]
```



Use `$merge` for incremental updates, `$out` for full rebuilds. If you re-run a daily aggregation, `$merge` with `whenMatched: "replace"` updates today's row without touching yesterday's. `$out` would wipe the entire collection.

17

\$count

Count documents — as a pipeline stage, not a method

Aggregation BACKEND WORKSHOP

Without it: you call `.length` on the result array in JS. That means every document was fetched over the network just to be counted. You shipped data to count it.

What it does: counts the number of documents at this point in the pipeline and returns a single document with the count in the field name you specify. The pipeline ends here — there's nothing left to count after.

EXAMPLE 1 — COUNT VERIFIED USERS

mongosh

copy

```
db.users.aggregate([
  { $match: { isVerified: true } },
  { $count: "verifiedCount" }
])
```

Output — 1 document

```
[ { verifiedCount: 5 } ]
```

EXAMPLE 2 — COUNT POSTS WITH 500+ LIKES

mongosh

copy

```
db.posts.aggregate([
  { $match: { likesCount: { $gt: 500 } } },
  { $count: "popularPosts" }
])
```

Output — posts with >500 likes: p2(890), p3(1200), p6(560), p7(1450), p8(780), p9(2100), p10(670), p11(950)

```
[ { popularPosts: 8 } ]
```

Randomly select N documents from the pipeline

Without it: you fetch all documents, shuffle the array in JS with Fisher–Yates, and slice. Every document crosses the network so you can throw most of them away.

What it does: randomly selects up to N documents from the current pipeline input. The selection is pseudo-random — MongoDB uses a sampling algorithm internally. Output order is not guaranteed.

EXAMPLE — PICK 3 RANDOM POSTS

mongosh

copy

```
db.posts.aggregate([
  { $sample: { size: 3 } },
  { $project: { caption: 1, _id: 0 } }
])
```

Output — non-deterministic, example run:

```
[
  { caption: "5AM workout hits different" },
  { caption: "New charcoal sketch, took 6 hours" },
  { caption: "MongoDB aggregation is actually beautiful" }
]
// Your output WILL differ — these are random picks.
```



\$sample is great for "featured post" carousels or A/B test bucketing. If the pipeline input is small (like our 12 posts), MongoDB may scan all of them. For large collections, it uses a more efficient sampling strategy.

19

\$facet

Run multiple sub-pipelines in parallel on the same input

Without it: your dashboard needs (a) posts grouped by category, (b) top 3 liked posts, and (c) total post count. You run three separate aggregation queries. Three round trips and the data might change

Aggregation BACKEND WORKSHOP

between queries.

What it does: takes the current pipeline input and fans it out into multiple named sub-pipelines. Each sub-pipeline runs independently on a copy of the same data. The output is a single document with one field per sub-pipeline, each containing that pipeline's results.

EXAMPLE – DASHBOARD: CATEGORY COUNTS + TOP POSTS + TOTAL COUNT IN ONE QUERY

mongosh

copy

```
db.posts.aggregate([
  { $facet: {
    "byCategory": [
      { $group: { _id: "$category", count: { $sum: 1 } } },
      { $sort: { count: -1 } }
    ],
    "topLiked": [
      { $sort: { likesCount: -1 } },
      { $limit: 3 },
      { $project: { caption: 1, likesCount: 1, _id: 0 } }
    ],
    "totalCount": [
      { $count: "count" }
    ]
  }
}]
```

Output – 1 document with 3 arrays

```
[
  {
    byCategory: [
      { _id: "tech",    count: 3 },
      { _id: "travel", count: 3 },
      { _id: "food",   count: 2 },
      { _id: "fitness", count: 2 },
      { _id: "art",    count: 2 }
    ],
    topLiked: [
      { caption: "Street food tour of Indore, thread", likesCount: 2100 },
      { caption: "Road trip through Spiti valley",      likesCount: 1450 },
      { caption: "Best butter chicken in Indore...",   likesCount: 1200 }
    ],
    totalCount: [ { count: 12 } ]
  }
]
```

Aggregation BACKEND WORKSHOP

```
}  
]
```

💡 Each sub-pipeline is independent — they don't see each other's results. The input to each sub-pipeline is the documents that entered `$facet`. Put `$match` before `$facet` so all sub-pipelines operate on the same filtered set.

20 \$bucket

Group documents into ranges you define manually

Without it: you fetch all users, then write if/else chains in JS to categorize each one into age buckets (20–22, 23–25, 26–29), maintaining count arrays per bucket.

What it does: categorizes documents into groups based on a specified field's value, using explicit boundary values you provide. Each bucket is [lower, upper) — lower inclusive, upper exclusive — except the last, which is inclusive on both ends.

EXAMPLE — BUCKET USERS BY AGE

mongosh

copy

```
db.users.aggregate([  
  { $bucket: {  
    groupBy: "$age",  
    boundaries: [20, 23, 26, 30],  
    default: "30+",  
    output: {  
      count: { $sum: 1 },  
      names: { $push: "$name" }  
    }  
  }  
}]  
)
```

Output — 3 buckets (no users fall into "30+")

```
[
  {
    _id: 20,
    count: 2,
    names: [ "Aarav Sharma", "Vihaan Gupta" ]
    // u1 (age 22), u5 (age 21) → bucket [20, 23)
  },
  {
    _id: 23,
    count: 3,
    names: [ "Ishita Verma", "Ananya Rao", "Diya Patel" ]
    // u6 (age 23), u4 (age 24), u2 (age 25) → bucket [23, 26)
  },
  {
    _id: 26,
    count: 3,
    names: [ "Reyansh Joshi", "Myra Kapoor", "Kabir Singh" ]
    // u7 (age 26), u8 (age 27), u3 (age 28) → bucket [26, 30]
  }
]
```



The `_id` of each output document is the lower boundary of the bucket. The `default` field catches any documents that don't fall into any defined bucket — if no documents fall outside, the default bucket is omitted from output.

21

\$bucketAuto

Group documents into N roughly equal-sized buckets — MongoDB picks boundaries

Without it: you want to split users into 3 follower tiers, but you don't know where the natural breaks are. You'd sort all users by followersCount, divide into thirds, and compute the boundaries yourself.

What it does: automatically computes boundary values to distribute documents into approximately N equal-sized buckets. You specify the number of buckets; MongoDB figures out where to draw the lines.

EXAMPLE — AUTO-BUCKET USERS INTO 3 GROUPS BY FOLLOWERS

```
db.users.aggregate([
  { $bucketAuto: {
    groupBy: "$followersCount",
    buckets: 3,
    output: {
      count: { $sum: 1 },
      names: { $push: "$name" }
    }
  }}
])
```

Output – 3 buckets (sorted by followers: 150, 320, 600, 1200, 2100, 3300, 4500, 8900)

```
[
  {
    _id: { min: 150, max: 1200 },
    count: 3,
    names: [ "Vihaan Gupta", "Kabir Singh", "Reyansh Joshi" ]
    // followers: 150, 320, 600
  },
  {
    _id: { min: 1200, max: 3300 },
    count: 2,
    names: [ "Aarav Sharma", "Ishita Verma" ]
    // followers: 1200, 2100
  },
  {
    _id: { min: 3300, max: 8900 },
    count: 3,
    names: [ "Myra Kapoor", "Diya Patel", "Ananya Rao" ]
    // followers: 3300, 4500, 8900
  }
]
```



Buckets may not be exactly equal. MongoDB distributes documents as evenly as it can, but if many documents share the same value, they all go into the same bucket. With 8 documents and 3 buckets, we get a 3-2-3 split. The `_id` now contains both `min` and `max` boundaries.

22

Operators — Accumulators

Used inside `$group` to aggregate values across documents in each group

Accumulators are the operators that go inside `$group`'s field definitions. They take a value from each document in the group and reduce them to a single output value. You've already seen `$sum` and `$avg` — here are all of them.

`$SUM`

Adds up numeric values across all documents in the group. Pass `1` to count documents.

mongosh

copy

```
db.users.aggregate([
  { $group: { _id: "$city", totalFollowers: { $sum: "$followersCount" } } },
  { $sort: { totalFollowers: -1 } }
])
```

Output

```
[
  { _id: "Indore", totalFollowers: 16700 }, // 4500 + 8900 + 3300
  { _id: "Bhopal", totalFollowers: 3620 }, // 1200 + 320 + 2100
  { _id: "Pune", totalFollowers: 750 } // 150 + 600
]
```

`$AVG`

Computes the arithmetic mean of numeric values across the group.

mongosh

copy

```
db.users.aggregate([
  { $group: { _id: "$city", avgFollowers: { $avg: "$followersCount" } } },
  { $sort: { avgFollowers: -1 } }
])
```

Output

```
[
  { _id: "Indore", avgFollowers: 5566.67 }, // 16700 / 3
  { _id: "Bhopal", avgFollowers: 1206.67 }, // 3620 / 3
  { _id: "Pune", avgFollowers: 375 } // 750 / 2
]
```

\$MIN / \$MAX

Returns the minimum / maximum value of the specified field across the group.

mongosh

copy

```
db.posts.aggregate([
  { $group: {
    _id: "$category",
    minLikes: { $min: "$likesCount" },
    maxLikes: { $max: "$likesCount" }
  }},
  { $sort: { _id: 1 } }
])
```

Output

```
[
  { _id: "art", minLikes: 310, maxLikes: 430 }, // p12(310), p5(430)
  { _id: "fitness", minLikes: 210, maxLikes: 780 }, // p4(210), p8(780)
  { _id: "food", minLikes: 1200, maxLikes: 2100 }, // p3(1200), p9(2100)
  { _id: "tech", minLikes: 340, maxLikes: 670 }, // p1(340), p10(670)
  { _id: "travel", minLikes: 890, maxLikes: 1450 } // p2(890), p7(1450)
]
```

\$PUSH

Collects a value from every document in the group into an array — no de-duplication.

mongosh

copy

```
db.comments.aggregate([
  { $group: { _id: "$postId", commentTexts: { $push: "$text" } } },
  { $match: { _id: "p3" } }
])
```

Aggregation BACKEND WORKSHOP

Output

```
[
  {
    _id: "p3",
    commentTexts: [
      "Ok now I'm hungry",
      "Which place exactly?",
      "Facts, best in the city",
      "Adding to my list"
    ]
  }
]
// c6, c7, c8, c9 - p3's 4 comments, in insertion order
```

\$ADDTOSET

Like `$push`, but drops duplicate values — every element in the output array is unique.

mongosh

copy

```
db.posts.aggregate([
  { $group: { _id: "$userId", categoriesPosted: { $addToSet: "$category" } } },
  { $match: { _id: "u1" } }
])
```

Output

```
[
  { _id: "u1", categoriesPosted: [ "tech" ] }
]
// u1 has p1 and p10, both category "tech" → deduplicated to one entry
```

\$FIRST / \$LAST

Returns the value from the first / last document in each group, based on the document order the group received them in (usually you'll `$sort` before `$group` to make this meaningful).

mongosh

copy

```
db.posts.aggregate([
  { $sort: { category: 1, createdAt: 1 } },
  { $group: {
```

Aggregation BACKEND WORKSHOP

```

    _id: "$category",
    earliestPost: { $first: "$caption" },
    latestPost: { $last: "$caption" }
  }}
])

```

Output

```

[
  { _id: "art",    earliestPost: "New charcoal sketch, took 6 hours", latestPost: "Di",
    { _id: "fitness", earliestPost: "Leg day never lies",          latestPost: "5AM",
    { _id: "food",   earliestPost: "Best butter chicken in Indore...", latestPost: "St",
    { _id: "tech",   earliestPost: "Deployed my first MERN app today!", latestPost: "M",
    { _id: "travel", earliestPost: "Sunset in Manali, worth every step", latestPost: "
]

```

\$COUNT (ACCUMULATOR)

MongoDB 5.0+ shorthand for `{ $sum: 1 }` inside `$group` — counts documents in each group without needing a field reference.

mongosh

copy

```

db.comments.aggregate([
  { $group: { _id: "$postId", numComments: { $count: {} } } },
  { $sort: { numComments: -1 } },
  { $limit: 3 }
])

```

Output

```

[
  { _id: "p3", numComments: 4 }, // c6, c7, c8, c9
  { _id: "p1", numComments: 3 }, // c1, c2, c3
  { _id: "p6", numComments: 3 }  // c13, c14, c15
]

```

\$STDDEVPOP / \$STDDEVSAMP

Population / sample standard deviation of a numeric field across the group. Use `$stdDevPop` when your group IS the entire population; `$stdDevSamp` when your group is a subset.

Aggregation BACKEND WORKSHOP

mongosh

copy

```
db.posts.aggregate([
  { $group: {
    _id: null,
    avgLikes: { $avg: "$likesCount" },
    likesStdDev: { $stdDevPop: "$likesCount" }
  }}
])
```

Output

```
[
  { _id: null, avgLikes: 898.33, likesStdDev: 508.7 }
  // _id: null groups ALL 12 posts into a single group
  // high stdDev reflects the spread: likes range from 210 to 2100
]
```

OPERATOR	RETURNS	COMMON USE
<code>\$sum</code>	Total of a numeric field, or document count	Totals, counts
<code>\$avg</code>	Arithmetic mean	Averages, ratings
<code>\$min</code> / <code>\$max</code>	Smallest / largest value	Ranges, leaderboards
<code>\$push</code>	Array of all values (with duplicates)	Collecting related items
<code>\$addToSet</code>	Array of unique values	Distinct tags, categories
<code>\$first</code> / <code>\$last</code>	Value from first / last doc in group order	Earliest/latest record per group
<code>\$count</code>	Document count in the group	Shorthand for { <code>\$sum</code> : 1 }
<code>\$stdDevPop</code> / <code>\$stdDevSamp</code>	Standard deviation	Spread / variance analysis

Operators — Arithmetic

Math on field values, used inside `$project`, `$addFields`, `$group`

Without them: you fetch raw numbers and do arithmetic in JS after the fact — meaning any computed value used for sorting or filtering has to happen client-side too, since the database never saw the derived number.

`$ADD` / `$SUBTRACT` / `$MULTIPLY` / `$DIVIDE` / `$MOD`

Basic arithmetic. `$add` and `$multiply` take an array of 2+ values; `$subtract`, `$divide`, `$mod` take exactly 2 (in order — first minus/divided-by second).

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p3" } },
  { $project: {
    caption: 1,
    totalEngagement: { $add: ["$likesCount", "$commentsCount"] },
    likesPerComment: { $divide: ["$likesCount", "$commentsCount"] },
    doubledLikes: { $multiply: ["$likesCount", 2] },
    likesMod100: { $mod: ["$likesCount", 100] }
  }}
])
```

Output — p3: likesCount 1200, commentsCount 4

```
[
  {
    caption: "Best butter chicken in Indore, change my mind",
    totalEngagement: 1204,    // 1200 + 4
    likesPerComment: 300,    // 1200 / 4
    doubledLikes: 2400,    // 1200 * 2
    likesMod100: 0          // 1200 % 100
  }
]
```

`$ROUND` / `$CEIL` / `$FLOOR` / `$TRUNC`

Rounding controls. `$round` rounds to nearest (optionally to N decimal places), `$ceil` always rounds up, `$floor` always rounds down, `$trunc` chops decimals without rounding.

mongosh

copy

```
db.posts.aggregate([
  { $match: { category: "food" } },
  { $group: { _id: "$category", avgLikes: { $avg: "$likesCount" } } },
  { $project: {
    avgLikes: 1,
    rounded: { $round: ["$avgLikes", 1] },
    ceiling: { $ceil: "$avgLikes" },
    floor: { $floor: "$avgLikes" }
  }}
])
```

Output — food avgLikes = (1200 + 2100) / 2 = 1650 (already whole)

```
[
  { _id: "food", avgLikes: 1650, rounded: 1650, ceiling: 1650, floor: 1650 }
]
// try this on "travel" instead — avgLikes 1096.666...:
// rounded: 1096.7, ceiling: 1097, floor: 1096, trunc: 1096
```

\$ABS

Returns the absolute value of a number — negatives become positive.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: { $in: ["p9", "p4"] } } },
  { $project: {
    caption: 1,
    likesGapFromAvg: { $abs: { $subtract: ["$likesCount", 898.33] } }
  }}
])
```

Output — 898.33 is the overall avg likesCount across all 12 posts

```
[
  { caption: "Street food tour of Indore, thread", likesGapFromAvg: 1201.67 }, // |21
```

```
{ caption: "Leg day never lies", likesGapFromAvg: 688.33 } // /2  
]
```

24

Operators — Comparison

Compare two values inside an expression — return true/false or -1/0/1

Without them: comparisons inside \$match use query operators like `$gt`, but inside \$project/\$addFields expressions you need the aggregation-expression versions of the same comparisons to build computed boolean fields.

`$EQ` / `$NE` / `$GT` / `$GTE` / `$LT` / `$LTE`

Standard comparisons as expressions — each takes a 2-element array `[value1, value2]` and returns a boolean.

mongosh

copy

```
db.users.aggregate([  
  { $project: {  
    name: 1,  
    followersCount: 1,  
    isInfluencer: { $gte: ["$followersCount", 3000] },  
    isNewbie: { $lt: ["$followersCount", 500] },  
    _id: 0  
  }}  
)
```

Output — 8 documents

```
[  
  { name: "Aarav Sharma", followersCount: 1200, isInfluencer: false, isNewbie: false  
  { name: "Diya Patel", followersCount: 4500, isInfluencer: true, isNewbie: false  
  { name: "Kabir Singh", followersCount: 320, isInfluencer: false, isNewbie: true  
  { name: "Ananya Rao", followersCount: 8900, isInfluencer: true, isNewbie: false  
  { name: "Vihaan Gupta", followersCount: 150, isInfluencer: false, isNewbie: true  
  { name: "Ishita Verma", followersCount: 2100, isInfluencer: true, isNewbie: false  
  { name: "Reyansh Joshi", followersCount: 600, isInfluencer: false, isNewbie: false  
]
```

Aggregation BACKEND WORKSHOP

```
{ name: "Myra Kapoor", followersCount: 3300, isInfluencer: true, isNewbie: false }]
```

\$CMP

Three-way comparison. Returns `-1` if first < second, `0` if equal, `1` if first > second. Rarely used directly — mostly informative for understanding how MongoDB sorts internally.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: { $in: ["p1", "p2"] } } },
  { $project: {
    caption: 1,
    vsThreshold: { $cmp: ["$likesCount", 500] }
  }}
])
```

Output

```
[
  { caption: "Deployed my first MERN app today!", vsThreshold: -1 }, // 340 < 500
  { caption: "Sunset in Manali, worth every step", vsThreshold: 1 } // 890 > 500
]
```

25

Operators — Logical / Conditional

Branch and combine boolean logic inside expressions

Without them: any "if this then that" logic on computed fields would require pulling documents into JS first, since `$project/$addFields` expressions have no native if-statement — these operators are how you branch inside the pipeline itself.

`$AND` / `$OR` / `$NOT`

Boolean combinators. `$and` / `$or` take an array of expressions; `$not` takes a single expression (wrapped in an array) and inverts it.

Aggregation BACKEND WORKSHOP

mongosh

copy

```
db.users.aggregate([
  { $project: {
    name: 1,
    risingStar: {
      $and: [
        { $eq: ["$isVerified", false] },
        { $gte: ["$followersCount", 300] }
      ]
    },
    _id: 0
  }}
])
```

Output – unverified users with 300+ followers

```
[
  { name: "Diya Patel",    risingStar: false }, // verified
  { name: "Kabir Singh",  risingStar: true  }, // unverified, 320 followers
  { name: "Vihaan Gupta", risingStar: false }, // unverified but only 150
  { name: "Reyansh Joshi", risingStar: true  }, // unverified, 600 followers
  // ... remaining verified users all false
]
```

\$COND

Inline if/else. Takes `{ if, then, else }` or a 3-element array `[condition, thenValue, elseValue]`.

mongosh

copy

```
db.posts.aggregate([
  { $project: {
    caption: 1,
    likesCount: 1,
    status: {
      $cond: {
        if: { $gte: ["$likesCount", 1000] },
        then: "viral",
        else: "normal"
      }
    }
  }}
])
```

```
  }}  
  ])
```

Output — posts with 1000+ likes: p3, p7, p9

```
[  
  { caption: "Deployed my first MERN app today!",      likesCount: 340, status:  
    { caption: "Sunset in Manali, worth every step",    likesCount: 890, status:  
      { caption: "Best butter chicken in Indore, change my mind", likesCount: 1200, status:  
        // ... p7 (1450) and p9 (2100) also "viral", rest "normal"  
      }  
    }  
  ]
```

\$IFNULL

Returns the first non-null/non-missing expression. Common for supplying default values when a field might not exist on some documents.

mongosh

copy

```
db.posts.aggregate([  
  { $match: { _id: "p7" } },  
  { $project: {  
    caption: 1,  
    commentsCount: 1,  
    displayComments: { $ifNull: ["$commentsCount", "No comments yet"] }  
  }}  
])
```

Output — p7 has commentsCount: 0, which is NOT null, so it passes through

```
[  
  { caption: "Road trip through Spiti valley", commentsCount: 0, displayComments: 0 }  
]  
  
// $ifNull only substitutes on null/missing, not on falsy values like 0.  
// This trips up beginners who expect it to catch 0 or empty string too.
```

\$SWITCH

Multi-branch conditional — cleaner than nesting multiple `$cond`. Evaluates `branches` in order, uses `default` if none match.

Aggregation BACKEND WORKSHOP

```
db.posts.aggregate([
  { $project: {
    caption: 1,
    likesCount: 1,
    tier: {
      $switch: {
        branches: [
          { case: { $gte: ["$likesCount", 1500] }, then: "S" },
          { case: { $gte: ["$likesCount", 800] }, then: "A" },
          { case: { $gte: ["$likesCount", 400] }, then: "B" }
        ],
        default: "C"
      }
    }
  } },
  { $sort: { likesCount: -1 } }
])
```

Output — first 4, sorted by likes descending

```
[
  { caption: "Street food tour of Indore, thread", likesCount: 2100, tier: "S" },
  { caption: "Road trip through Spiti valley",      likesCount: 1450, tier: "A" },
  { caption: "Best butter chicken in Indore...",    likesCount: 1200, tier: "A" },
  { caption: "Backwaters of Kerala at dawn",        likesCount: 950, tier: "A" }
]
```

26

Operators — String

Manipulate text fields inside the pipeline

Without them: formatting, searching, or transforming text (capitalization, trimming, splitting usernames) all had to happen in JS after every document was fetched — you couldn't filter or sort on the transformed value at the database level.

Joins multiple strings into one. All arguments must resolve to strings — no automatic number-to-string conversion.

mongosh

copy

```
db.users.aggregate([
  { $match: { _id: "u1" } },
  { $project: {
    handle: { $concat: ["@", "$username", " . ", "$city"] },
    _id: 0
  }}
])
```

Output

```
[ { handle: "@aarav_codes . Bhopal" } ]
```

\$TOUPPER / \$TOLOWER

Case conversion.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p4" } },
  { $project: {
    shout: { $toUpper: "$caption" },
    categoryLower: { $toLower: "$category" },
    _id: 0
  }}
])
```

Output

```
[ { shout: "LEG DAY NEVER LIES", categoryLower: "fitness" } ]
```

\$TRIM

Strips whitespace (or specified characters) from both ends of a string. All our sample captions are already clean, so here's a synthetic value to show the behavior.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p1" } },
  { $project: {
    cleaned: { $trim: { input: "  $caption  " } },
    _id: 0
  }}
])
// note: input must be an expression, this shows the shape – in real use
// you'd trim a field that actually has padding, e.g. user-submitted text
```

\$SUBSTR / \$SUBSTRBYTES

Extracts a portion of a string given a start index and length. `$substrBytes` is byte-based (safe for ASCII); prefer `$substrCP` for multi-byte/unicode-safe extraction in modern MongoDB.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p1" } },
  { $project: {
    preview: { $substrBytes: ["$caption", 0, 20] },
    _id: 0
  }}
])
```

Output – first 20 characters of "Deployed my first MERN app today!"

```
[ { preview: "Deployed my first ME" } ]
```

\$SPLIT

Splits a string into an array using a delimiter.

mongosh

copy

```
db.users.aggregate([
  { $match: { _id: "u4" } },
  { $project: {
    usernameParts: { $split: ["$username", "."] },
    _id: 0
  }}
])
```

Aggregation BACKEND WORKSHOP

```
  }}  
  ])
```

Output – u4's username is "ananya.eats"

```
[ { usernameParts: [ "ananya", "eats" ] } ]
```

\$STRLENCP

Length of a string in Unicode code points (safe for counting characters, including multi-byte ones).

mongosh

copy

```
db.posts.aggregate([  
  { $project: {  
    caption: 1,  
    captionLength: { $strLenCP: "$caption" },  
    _id: 0  
  }},  
  { $sort: { captionLength: -1 } },  
  { $limit: 1 }  
])
```

Output – longest caption

```
[ { caption: "Best butter chicken in Indore, change my mind", captionLength: 46 } ]
```

\$STRCASECMP

Case-insensitive string comparison. Returns -1, 0, or 1 like `$cmp`, but ignores case.

mongosh

copy

```
db.posts.aggregate([  
  { $match: { _id: "p6" } },  
  { $project: {  
    matchesTech: { $strcasecmp: ["$category", "TECH"] },  
    _id: 0  
  }},  
  { $limit: 1 }  
])
```

Output – "tech" vs "TECH" are equal, case-insensitively

```
[ { matchesTech: 0 } ]
```

\$REGEXMATCH / \$REGEXFIND / \$REGEXFINDALL

`$regexMatch` returns a boolean. `$regexFind` returns the first match with its position. `$regexFindAll` returns every match.

mongosh

copy

```
db.posts.aggregate([
  { $project: {
    caption: 1,
    mentionsIndore: {
      $regexMatch: { input: "$caption", regex: "Indore", options: "i" }
    },
    _id: 0
  }},
  { $match: { mentionsIndore: true } }
])
```

Output – captions matching "Indore" (case-insensitive): p3, p9

```
[
  { caption: "Best butter chicken in Indore, change my mind", mentionsIndore: true },
  { caption: "Street food tour of Indore, thread", mentionsIndore: true }
]
```

OPERATOR

PURPOSE

`$concat`

Join strings together

`$toUpper` / `$toLower`

Change case

`$trim`

Remove leading/trailing characters

`$substr` / `$substrBytes`

Extract a portion of a string

`$split`

Break a string into an array

`$strLenCP`

Aggregation BACKEND WORKSHOP
Character count

27

Operators — Array

Work with array fields without needing \$unwind

Without them: to check array length, filter array contents, or transform elements, you'd fetch the document and manipulate the array in JS — meaning you can't sort or filter documents by a derived array property at the database level.

\$SIZE

Returns the number of elements in an array. You already saw this with `$lookup` results.

mongosh

copy

```
db.posts.aggregate([
  { $project: { caption: 1, tagCount: { $size: "$tags" }, _id: 0 } },
  { $sort: { tagCount: -1 } },
  { $limit: 3 }
])
```

Output

```
[
  { caption: "Deployed my first MERN app today!",      tagCount: 3 },
  { caption: "Sunset in Manali, worth every step",     tagCount: 3 },
  { caption: "Best butter chicken in Indore, change my mind", tagCount: 3 }
]
```

\$ARRAYELEMAT

Returns the element at a given index. Negative indices count from the end.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p1" } },
  { $project: {
    firstTag: { $arrayElemAt: ["$tags", 0] },
    lastTag: { $arrayElemAt: ["$tags", -1] },
    _id: 0
  }}
])
```

Output — p1 tags: ["mern", "webdev", "nodejs"]

```
[ { firstTag: "mern", lastTag: "nodejs" } ]
```

\$SLICE

Returns a subset of an array — pass `[array, n]` for the first n elements, or `[array, start, n]` for n elements starting at an index.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p1" } },
  { $project: {
    firstTwoTags: { $slice: ["$tags", 2] },
    _id: 0
  }}
])
```

Output

```
[ { firstTwoTags: [ "mern", "webdev" ] } ]
```

\$FILTER

Filters array elements against a condition, similar to JS `Array.filter()` but running inside the pipeline.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p3" } },
  { $project: {
```

Aggregation BACKEND WORKSHOP

```

    longTagsOnly: {
      $filter: {
        input: "$tags",
        as: "tag",
        cond: { $gt: [{ $strLenCP: "$$tag" }, 5] }
      }
    },
    _id: 0
  }
})

```

Output – p3 tags: ["food", "indore", "foodie"], only those over 5 chars

```
[ { longTagsOnly: [ "indore", "foodie" ] } ]
```

\$MAP

Transforms every element of an array, similar to JS `Array.map()`.

mongosh

copy

```

db.posts.aggregate([
  { $match: { _id: "p1" } },
  { $project: {
    hashtags: {
      $map: {
        input: "$tags",
        as: "tag",
        in: { $concat: ["#", "$$tag"] }
      }
    },
    _id: 0
  }
})

```

Output

```
[ { hashtags: [ "#mern", "#webdev", "#nodejs" ] } ]
```

\$REDUCE

Folds an array down to a single value, accumulating as it goes — like JS `Array.reduce()`. Access the running total via `$$value` and the current element via `$$this`.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p1" } },
  { $project: {
    tagsJoined: {
      $reduce: {
        input: "$tags",
        initialValue: "",
        in: {
          $concat: [
            "$$value",
            { $cond: [{ $eq: ["$$value", ""] }, "", ", ", " ] },
            "$$this"
          ]
        }
      }
    }
  },
  _id: 0
}]
```

Output

```
[ { tagsJoined: "mern, webdev, nodejs" } ]
```

\$IN

Checks whether a value exists in an array. Returns a boolean.

mongosh

copy

```
db.posts.aggregate([
  { $project: {
    caption: 1,
    isWebdev: { $in: ["webdev", "$tags"] },
    _id: 0
  }},
  _id: 0
}]
```

 Aggregation BACKEND WORKSHOP

```
{ $match: { isWebdev: true } }  
])
```

Output – posts tagged "webdev": p1, p6, p10

```
[  
  { caption: "Deployed my first MERN app today!", isWebdev: true },  
  { caption: "Understanding closures finally clicked", isWebdev: true },  
  { caption: "MongoDB aggregation is actually beautiful", isWebdev: true }  
]
```

\$INDEXOFARRAY

Returns the index of the first matching element, or -1 if not found.

mongosh

copy

```
db.posts.aggregate([  
  { $match: { _id: "p3" } },  
  { $project: {  
    foodieIndex: { $indexOfArray: ["$tags", "foodie"] },  
    _id: 0  
  }}  
])
```

Output – p3 tags: ["food", "indore", "foodie"]

```
[ { foodieIndex: 2 } ]
```

\$CONCATARRAYS

Merges multiple arrays into one.

mongosh

copy

```
db.posts.aggregate([  
  { $match: { _id: { $in: ["p1", "p10"] } } },  
  { $group: { _id: "$userId", allTags: { $push: "$tags" } } },  
  { $project: {  
    combinedTags: { $reduce: { input: "$allTags", initialValue: [], in: { $concatArray
```

```
  }  
  ]  
}
```

Output – u1's tags from p1 ["mern","webdev","nodejs"] + p10
["mongodb","webdev","backend"]

```
[  
  {  
    _id: "u1",  
    combinedTags: [ "mern", "webdev", "nodejs", "mongodb", "webdev", "backend" ]  
    // note: "webdev" appears twice – $concatArrays doesn't dedupe, unlike $addToSet  
  }  
]
```

\$MERGEOBJECTS

Merges multiple documents into one — later documents' fields override earlier ones on conflict. You saw this with `$replaceRoot` earlier.

mongosh

copy

```
db.posts.aggregate([  
  { $match: { _id: "p1" } },  
  { $project: {  
    merged: {  
      $mergeObjects: [  
        { source: "user-generated", category: "$category" },  
        { category: "tech-verified" }  
      ]  
    },  
    _id: 0  
  } }  
])
```

Output – second object's category wins

```
[ { merged: { source: "user-generated", category: "tech-verified" } } ]
```

Operators — Date

Extract, format, and compute with date fields

Without them: "posts per month" or "users who joined this week" style reports require pulling every document and doing date math in JS — the database can't group by "month" without extracting it as a real value first.



Our sample dates are stored as strings (e.g. `"2024-06-01"`) for readability in these notes. In the actual codebase, these fields are real BSON `Date` objects — date operators require actual Date types, not strings, so the seed script converts them on insert.

`$YEAR` / `$MONTH` / `$DAYOFMONTH` / `$HOUR` / `$MINUTE` / `$DAYOFWEEK`

Extract individual components from a Date value.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p1" } },
  { $project: {
    year:    { $year:    "$createdAt" },
    month:   { $month:   "$createdAt" },
    day:     { $dayOfMonth: "$createdAt" },
    dayOfWeek: { $dayOfWeek: "$createdAt" },
    _id: 0
  }}
])
```

Output — p1 createdAt: 2024-06-01 (a Saturday)

```
[ { year: 2024, month: 6, day: 1, dayOfWeek: 7 } ]
// $dayOfWeek: 1 = Sunday, 7 = Saturday
```

`$DATEToString`

Formats a Date as a string using a format specifier, similar to `strftime`.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p1" } },
  { $project: {
    formatted: { $dateToString: { format: "%d %b %Y", date: "$createdAt" } },
    _id: 0
  }}
])
```

Output

```
[ { formatted: "01 Jun 2024" } ]
```

\$DATEFROMSTRING

Parses a string into a Date value — the inverse of `$dateToString`.

mongosh

copy

```
db.posts.aggregate([
  { $project: {
    parsedDate: { $dateFromString: { dateString: "2024-06-01" } },
    _id: 0
  }},
  { $limit: 1 }
])
```

Output

```
[ { parsedDate: ISODate("2024-06-01T00:00:00.000Z") } ]
```

\$DATEADD / \$DATESUBTRACT

Adds or subtracts a duration from a date. Useful for computing "posted within last N days" windows.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p1" } },
  { $project: {
    weekLater: { $dateAdd: { startDate: "$createdAt", unit: "day", amount: 7 } },
    _id: 0
  }}
])
```

```
  }}  
])
```

Output — p1 createdAt: 2024-06-01

```
[ { weekLater: ISODate("2024-06-08T00:00:00.000Z") } ]
```

\$DATEDIFF

Computes the difference between two dates in a specified unit.

mongosh

copy

```
db.posts.aggregate([  
  { $match: { _id: { $in: ["p1", "p12"] } } },  
  { $sort: { createdAt: 1 } },  
  { $group: { _id: null, first: { $first: "$createdAt" }, last: { $last: "$createdAt" } },  
  { $project: {  
    daysBetween: { $dateDiff: { startDate: "$first", endDate: "$last", unit: "day" } },  
    _id: 0  
  } }  
])
```

Output — p1: 2024-06-01, p12: 2024-06-16

```
[ { daysBetween: 15 } ]
```

\$DATETRUNC

Rounds a date down to the nearest unit (start of day, week, month, etc.) — the basis for "group posts by week" style reports.

mongosh

copy

```
db.posts.aggregate([  
  { $project: {  
    weekStart: { $dateTrunc: { date: "$createdAt", unit: "week" } }  
  } },  
  { $group: { _id: "$weekStart", postCount: { $sum: 1 } } },  
  { $sort: { _id: 1 } }  
])
```

Aggregation BACKEND WORKSHOP

Output — posts bucketed into their containing week

```
[
  { _id: ISODate("2024-05-26T00:00:00.000Z"), postCount: 4 }, // p1-p4 (Jun 1, 3, 5,
  { _id: ISODate("2024-06-02T00:00:00.000Z"), postCount: 4 }, // p5-p8 (Jun 7, 8, 10,
  { _id: ISODate("2024-06-09T00:00:00.000Z"), postCount: 4 } // p9-p12 (Jun 12, 14,
```

29

Operators — Type Conversion

Convert between BSON types explicitly

Without them: comparing or computing across mismatched types (a stringified number vs. a real number, for example) silently fails or produces wrong results in a pipeline — there's no implicit coercion the way JS does it.

`$TOSTRING` / `$TOINT` / `$TODOUBLE` / `$TODATE` / `$TOBOOL` / `$TOOBJECTID`

Direct conversions to each named type.

mongosh

copy

```
db.users.aggregate([
  { $match: { _id: "u1" } },
  { $project: {
    followersAsString: { $toString: "$followersCount" },
    ageAsDouble: { $toDouble: "$age" },
    _id: 0
  }}
])
```

Output

```
[ { followersAsString: "1200", ageAsDouble: 22.0 } ]
```

`$CONVERT`

Aggregation BACKEND WORKSHOP

Generic conversion — like the specific `$to*` operators, but lets you specify an `onError` and `onNull` fallback instead of throwing.

mongosh

copy

```
db.users.aggregate([
  { $match: { _id: "u1" } },
  { $project: {
    safeConvert: {
      $convert: { input: "$followersCount", to: "string", onError: "N/A", onNull: "unk
    },
    _id: 0
  }}
])
```

Output

```
[ { safeConvert: "1200" } ]
```

\$TYPE

Returns the BSON type name of a field's value as a string — useful for debugging inconsistent schemas.

mongosh

copy

```
db.users.aggregate([
  { $match: { _id: "u1" } },
  { $project: {
    followersType: { $type: "$followersCount" },
    isVerifiedType: { $type: "$isVerified" },
    _id: 0
  }}
])
```

Output

```
[ { followersType: "int", isVerifiedType: "bool" } ]
```

OPERATOR

CONVERTS TO

Aggregation BACKEND WORKSHOP

<code>\$toString</code>	String
<code>\$toInt</code>	32-bit integer
<code>\$toDouble</code>	Double-precision float
<code>\$toDate</code>	Date
<code>\$toBool</code>	Boolean
<code>\$toObjectId</code>	ObjectId
<code>\$convert</code>	Any type, with error/null fallbacks
<code>\$type</code>	N/A — inspects and returns the current type name

30

Operators — Variables

Define reusable values inside a single expression

Without them: if the same computed sub-expression is needed twice inside one pipeline stage, you'd have to repeat the entire expression — verbose, and if you need to change it, you change it in two places.

`$LET`

Defines one or more variables scoped to a single expression, avoiding repetition of a sub-computation.

mongosh

copy

```
db.posts.aggregate([
  { $match: { _id: "p3" } },
  { $project: {
    caption: 1,
    summary: {
      $let: {
        vars: { total: { $add: ["$likesCount", "$commentsCount"] } },
        in: { $concat: ["Total engagement: ", { $toString: "$$total" } ] }
      }
    },
  },
  _id: 0
```

Aggregation BACKEND WORKSHOP

```
    }}  
  ])
```

Output — p3: likesCount 1200 + commentsCount 4

```
[  
  {  
    caption: "Best butter chicken in Indore, change my mind",  
    summary: "Total engagement: 1204"  
  }  
]
```

\$LITERAL

Forces MongoDB to treat a value as literal, not as a field path or operator — needed when a literal string happens to start with `$` and you don't want it interpreted as a field reference.

mongosh

copy

```
db.posts.aggregate([  
  { $match: { _id: "p1" } },  
  { $project: {  
    literalDollarSign: { $literal: "$likesCount" }, // treated as the raw string, not  
    actualField: "$likesCount",                     // treated as a field reference  
    _id: 0  
  }}  
])
```

Output

```
[ { literalDollarSign: "$likesCount", actualField: 340 } ]
```

31

Combined Pipelines — Case Studies

This is the payoff. Stages and operators chained together to answer real questions

Aggregation BACKEND WORKSHOP

Everything above was one stage or operator at a time. Real pipelines chain 4–8 stages together. Here are three end-to-end examples on our exact dataset — each one answers a question you genuinely could not answer with a single `find()` call.

CASE STUDY 1

Most-commented post per category, with commenter names

"For each category, which post got the most discussion — and who was talking?"

This needs: group posts by category, join in comment counts, pick the winner per category, then join in the actual commenter names. Five stages, two different `$lookup` s.

mongosh

copy

```
db.posts.aggregate([
  // Stage 1: attach the actual comment documents to each post
  { $lookup: {
    from: "comments", localField: "_id", foreignField: "postId", as: "postComments"
  } },

  // Stage 2: compute a real comment count from the joined array
  { $addFields: { actualCommentCount: { $size: "$postComments" } } },

  // Stage 3: sort within each category by comment count, so $first grabs the winner
  { $sort: { category: 1, actualCommentCount: -1 } },

  // Stage 4: group by category, keep only the top post's data via $first
  { $group: {
    _id: "$category",
    caption: { $first: "$caption" },
    commentCount: { $first: "$actualCommentCount" },
    authorId: { $first: "$userId" },
    commenterIds: { $first: "$postComments.userId" }
  } },

  // Stage 5: join author details
  { $lookup: {
    from: "users", localField: "authorId", foreignField: "_id", as: "author"
  } },
  { $unwind: "$author" },

  // Stage 6: join commenter names
```

```

{ $lookup: {
  from: "users", localField: "commenterIds", foreignField: "_id", as: "commenters"
}},

{ $project: {
  _id: 0,
  category: "$_id",
  caption: 1,
  commentCount: 1,
  authorName: "$author.name",
  authorVerified: "$author.isVerified",
  commenterNames: "$commenters.name"
}},
{ $sort: { category: 1 } }
])

```

Output – 5 documents, one per category

```

[
  {
    category: "art",
    caption: "New charcoal sketch, took 6 hours",
    commentCount: 2,
    authorName: "Vihaan Gupta", authorVerified: false,
    commenterNames: [ "Aarav Sharma", "Reyansh Joshi" ]
  },
  {
    category: "fitness",
    caption: "Leg day never lies",
    commentCount: 1,
    authorName: "Kabir Singh", authorVerified: false,
    commenterNames: [ "Myra Kapoor" ]
  },
  {
    category: "food",
    caption: "Best butter chicken in Indore, change my mind",
    commentCount: 4,
    authorName: "Ananya Rao", authorVerified: true,
    commenterNames: [ "Aarav Sharma", "Myra Kapoor", "Ishita Verma", "Kabir Singh" ]
  },
  {
    category: "tech",
    caption: "Deployed my first MERN app today"
  }
]

```

```

    commentCount: 3,
    authorName: "Aarav Sharma", authorVerified: true,
    commenterNames: [ "Ishita Verma", "Kabir Singh", "Diya Patel" ]
  },
  {
    category: "travel",
    caption: "Sunset in Manali, worth every step",
    commentCount: 2,
    authorName: "Diya Patel", authorVerified: true,
    commenterNames: [ "Aarav Sharma", "Ananya Rao" ]
  }
]

```



Why `$first` after `$sort`, not `$group` directly: we sort by `actualCommentCount` descending within each category first, so when `$group` collapses each category, `$first` reliably grabs the highest-comment post's fields — not an arbitrary one.

CASE STUDY 2

City leaderboard — reach and top content per city

"Which city has the strongest community, and what's their best-performing post?"

This needs `$facet`-style thinking done manually with `$lookup` + `$group`: for each city, compute user stats, then separately find that city's best post, then merge the two.

mongosh

copy

```

db.users.aggregate([
  // Stage 1: bring in every post by each user
  { $lookup: {
    from: "posts", localField: "_id", foreignField: "userId", as: "posts"
  }},

  // Stage 2: flatten each user's posts into separate pipeline documents
  // (preserveNullAndEmptyArrays keeps users with zero posts in the pipeline)
  { $unwind: { path: "$posts", preserveNullAndEmptyArrays: true } },

  // Stage 3: group back by city, computing city-level stats

```

Aggregation BACKEND WORKSHOP

```

{ $group: {
  _id: "$city",
  userCount: { $addToSet: "$_id" },
  avgFollowers: { $avg: "$followersCount" },
  topPostLikes: { $max: "$posts.likesCount" },
  topPostCaption: {
    $first: {
      $cond: [{ $eq: ["$posts.likesCount", null] }, null, "$posts.caption"]
    }
  }
}},

{ $project: {
  _id: 0,
  city: "$_id",
  userCount: { $size: "$userCount" },
  avgFollowers: { $round: ["$avgFollowers", 2] },
  topPostLikes: 1
}},
{ $sort: { avgFollowers: -1 } }
])

```

Output — 3 documents

```

[
  { city: "Indore", userCount: 3, avgFollowers: 5566.67, topPostLikes: 2100 },
  { city: "Bhopal", userCount: 3, avgFollowers: 1206.67, topPostLikes: 670 },
  { city: "Pune", userCount: 2, avgFollowers: 375, topPostLikes: 950 }
]
// Indore wins on both reach and content performance:
// its top post is p9 "Street food tour of Indore, thread" at 2100 likes.

```

⚠️ **preserveNullAndEmptyArrays** matters here. Without it, a user with zero posts would vanish entirely from the pipeline after `$unwind` — silently skewing `avgFollowers` since that user's follower count would never reach the `$group` stage. Every one of our 8 users has at least one post, so this doesn't change today's output, but it's a bug waiting to happen the moment your data changes.

CASE STUDY 3

Most engaged commenters, with their favorite category

"Who comments the most on this platform, and what kind of content do they engage with?"

This needs: group comments by commenter, join back to posts to find categories they commented on, then find their most-frequent category — a `$lookup` followed by a nested group-and-rank.

mongosh

copy

```
db.comments.aggregate([
  // Stage 1: join each comment to the post it was made on
  { $lookup: {
    from: "posts", localField: "postId", foreignField: "_id", as: "post"
  }},
  { $unwind: "$post" },

  // Stage 2: group by commenter + category to count engagement per category
  { $group: {
    _id: { userId: "$userId", category: "$post.category" },
    categoryCount: { $sum: 1 }
  }},

  // Stage 3: sort so the top category per user comes first
  { $sort: { "_id.userId": 1, categoryCount: -1 } },

  // Stage 4: re-group by user, take total comments + top category via $first
  { $group: {
    _id: "$_id.userId",
    totalComments: { $sum: "$categoryCount" },
    favoriteCategory: { $first: "$_id.category" }
  }},

  // Stage 5: attach the user's name
  { $lookup: {
    from: "users", localField: "_id", foreignField: "_id", as: "user"
  }},
  { $unwind: "$user" },

  { $project: {
    _id: 0,
```

```

    name: "$user.name",
    totalComments: 1,
    favoriteCategory: 1
  }},
  { $sort: { totalComments: -1 } }
])

```

Output – 8 documents, most active commenter first

```

[
  { name: "Aarav Sharma", totalComments: 4, favoriteCategory: "tech" },
  { name: "Myra Kapoor", totalComments: 3, favoriteCategory: "food" },
  { name: "Ishita Verma", totalComments: 2, favoriteCategory: "tech" },
  { name: "Kabir Singh", totalComments: 2, favoriteCategory: "tech" },
  { name: "Diya Patel", totalComments: 1, favoriteCategory: "tech" },
  { name: "Ananya Rao", totalComments: 1, favoriteCategory: "travel" },
  { name: "Reyansh Joshi", totalComments: 1, favoriteCategory: "art" },
  { name: "Vihaan Gupta", totalComments: 1, favoriteCategory: "tech" }
]

```

WHY TWO \$GROUP STAGES

The first `$group` answers "how many times did this user comment on this category" — a fine-grained count. The second `$group` collapses those category-level counts back up to one row per user, using `$sum` to total all their comments and `$first` (after sorting) to grab their single most-commented category. This two-pass group pattern — group fine, sort, group coarse — is one of the most useful tricks in aggregation once you need a "top X per Y" answer.